Sentiment Analysis of Amazon Product Reviews

By Roy Meoded

Project Overview

This project focuses on sentiment analysis of Amazon product reviews, using classical machine learning methods and neural networks.
Sentiment analysis aims to automatically determine whether a given text expresses a positive or negative opinion.
Sentiment analysis is widely used in applications such as product reviews, social media monitoring, and customer feedback analysis.
It allows companies to automatically analyze large volumes of text data and extract meaningful insights.

Dataset Description

Dataset Source

The dataset used in this project was obtained from **Kaggle** and is based on Amazon product reviews.
The data is provided in **fastText** format, where each review is stored as a single line of text with an attached label.
Each label represents the sentiment of the review, either positive or negative.

fastText Data Format

In the fastText format, each line starts with a label token followed by the review text.
The label is written as **__label__1** or **__label__2**, indicating negative or positive sentiment.
During data loading, we parsed each line to separate the label from the review text and stored them in a structured **DataFrame**.

Data Loading & Preprocessing

After loading the dataset, the data was processed to prepare it for machine learning models.
Each review text was cleaned by converting all characters to lowercase and removing non-alphabetic symbols.
This preprocessing step helps reduce noise and ensures consistency in the input data.
The processed texts and their corresponding labels were stored in a **Pandas** DataFrame.
The DataFrame structure allows convenient data manipulation, analysis, and splitting into training and validation sets.
Using a structured DataFrame also improves code readability and reproducibility.

Baseline Models

Baseline models were implemented to provide simple reference points for evaluating more advanced models.
These models help determine whether complex machine learning approaches truly improve performance.
Without baselines, it would be difficult to assess the effectiveness of the final models.

Baseline 1 – Majority Class
The first baseline predicts the most frequent label in the dataset for all reviews.
In the dataset, **51,267** reviews were positive and **48,733** were negative, resulting in a majority positive class.
This baseline achieved an accuracy of **51.27%**, which serves as a minimal performance threshold.

Baseline 2 – Rule-Based Keywords
The second baseline applies a rule-based approach using predefined sentiment keywords.
This model achieved an accuracy of approximately **64.7%**, significantly outperforming the majority baseline.
However, its precision was limited (**~0.60**) due to misclassification of reviews containing ambiguous or missing keywords.

```
=== Baseline Results ===
+----------------+------------------+----------+
|    Baseline    |     Details      | Accuracy |
+----------------+------------------+----------+
| Majority Class |        1         |  0.5127  |
|   Rule-Based   | Keyword matching |  0.6473  |
+----------------+------------------+----------+
```

Baseline Comparison
The majority baseline achieved **51.99% accuracy** but a recall of **1.0**, as it predicted only one class.
The rule-based model improved performance to **64.92% accuracy** with an F1-score of **0.7434**.
These results demonstrate that while simple heuristics help, more expressive models are required for high performance.

Model	Accuracy	Precision	Recall	F1-score
Baseline 1 (Majority)	0.5199	0.5199	1.0000	0.6841
Baseline 2 (Rule-Based)	0.6492	0.5998	0.9774	0.7434

Text Representation

TF-IDF Vectorization
To convert textual reviews into numerical features, we applied the **TF-IDF** (Term Frequency–Inverse Document Frequency) representation.
Each review was transformed into a vector of **10,000 features**, including both unigrams and bigrams, while removing English stop words.
The training set contained **80,000 samples**, resulting in a TF-IDF matrix of shape **(80,000 × 10,000)**.

Logistic Regression Model

A Logistic Regression classifier was trained using the TF-IDF feature vectors.
The model was trained on 80% of the dataset and evaluated on the remaining 20% validation set consisting of 20,000 reviews.

Split	Size
Train	80000
Validation	20000

This model achieved an accuracy of 88.51%, demonstrating strong performance for a linear classifier.
The Logistic Regression model achieved a precision of **0.8859** and a recall of **0.8940**.
Its F1-score was **0.8900**, indicating a balanced trade-off between precision and recall.
These results show that TF-IDF combined with Logistic Regression is highly effective for sentiment analysis.

Model	Accuracy	Precision	Recall	F1-score
Baseline 1 (Majority)	0.5199	0.5199	1.0000	0.6841
Baseline 2 (Rule-Based)	0.6492	0.5998	0.9774	0.7434
Logistic Regression (TF-IDF)	0.8851	0.8859	0.8940	0.8900

Transition to Neural Networks

Although Logistic Regression achieved strong results, it remains a linear model.
To explore whether a non-linear model could further improve performance, we implemented a neural network using **PyTorch**.
This allowed us to compare classical machine learning with deep learning under the same feature representation.

Feed-Forward Neural Network Design

We implemented a Feed-Forward Neural Network (FFNN) using PyTorch to perform binary sentiment classification.
The network consists of an input layer with 10,000 neurons, a hidden layer of 128 neurons, a ReLU activation function, and a Dropout layer with a rate of 0.2.
The output layer contains a single neuron that produces a logit value for binary classification.

Loss Function and Optimizer

The model was trained using the **BCEWithLogitsLoss** loss function, which is suitable for binary classification tasks.
We used the **Adam optimizer** with a learning rate of **0.001** to update the model parameters efficiently.
Training was performed for **5 epochs** with a batch size of **256 samples**.

Learning Progress Across Epochs

During training, the average loss consistently decreased across epochs, indicating effective learning.
The training loss dropped from **0.3934** in the first epoch to **0.1967** in the fifth epoch.
This steady reduction shows that the neural network successfully learned meaningful patterns from the TF-IDF features.

Neural Network Evaluation

Validation Performance

After training, the neural network was evaluated on a validation set of **20,000** reviews.
The model achieved an accuracy of **88.00%**, with a precision of **0.8778** and a recall of **0.8937**.
The resulting F1-score was **0.8857**, showing performance comparable to Logistic Regression.

Model	Accuracy	Precision	Recall	F1-score
Baseline 1 (Majority)	0.5199	0.5199	1.0000	0.6841
Baseline 2 (Rule-Based)	0.6492	0.5998	0.9774	0.7434
Logistic Regression (TF-IDF)	0.8851	0.8859	0.8940	0.8900
PyTorch FFNN (TF-IDF)	0.8800	0.8778	0.8937	0.8857

Model Comparison

Both Logistic Regression and the neural network achieved very similar performance results.
This indicates that TF-IDF features already capture most of the useful information for this task.
The neural network did not significantly outperform the linear model, but confirmed the robustness of the feature representation.

Final Summary and Conclusions

In this project, we compared baseline methods, Logistic Regression, and a neural network for sentiment analysis.
The best performance was achieved using TF-IDF features combined with Logistic Regression, with an F1-score of **0.8900**.
These results demonstrate that classical machine learning methods can be highly effective for text classification tasks.